

Examen Final

Durée : 3H

Projet TypeScript — Gestion d'une Bibliothèque



Objectif :

une mini-application en console, structurée en modules TypeScript, qui simule la gestion d'une bibliothèque : livres, auteurs, utilisateurs, emprunts et retours.

Vous devez démontrer votre maîtrise de :

- Types de base, unions, alias
- Interfaces
- Fonctions typées et fléchées
- Classes, héritage, modificateurs d'accès, encapsulation
- Types avancés
- Modules
- Génériques (Repository)

Aucun framework requis.

Problématique du projet

Dans le cadre de la modernisation d'un système de gestion documentaire, la bibliothèque universitaire souhaite disposer d'un prototype applicatif lui permettant de gérer efficacement ses ouvrages et leurs emprunts.

L'objectif est de proposer une **modélisation complète**, ainsi qu'une **implémentation fonctionnelle** en TypeScript, capable de représenter les données essentielles (livres, auteurs, utilisateurs) et de simuler les processus réels d'une bibliothèque : ajout d'ouvrages, emprunts, retours, disponibilité, et suivi des utilisateurs.

Le système actuel ne permet ni de structurer correctement les entités métier, ni d'appliquer des règles de gestion simples (limite d'emprunts par étudiant, retard, réservation). Votre groupe est donc chargé de **construire un mini-système cohérent, typé et modulaire**, capable de répondre aux besoins suivants :

1. Le système doit représenter les auteurs, les livres, les utilisateurs et les emprunts.

Cela impose l'utilisation de :

- Interfaces pour les entités statiques (Author, Book, Loan),
- Classes et héritage pour les utilisateurs (User, Student, Librarian) et la gestion (Library),
- Types avancés pour formaliser les rôles (Role) et les statuts d'emprunt (LoanStatus).

2. Le système doit permettre :

- l'ajout de nouveaux ouvrages,
- l'affichage complet du catalogue,
- l'identification rapide des livres disponibles (non empruntés).

Ces fonctionnalités doivent être intégrées à la classe Library.

3. Le projet doit répondre aux questions suivantes :

- Comment enregistrer un nouvel emprunt avec un statut initial "ongoing" ?
- Comment effectuer un retour en mettant à jour l'état du livre et du prêt ?
- Comment faire respecter une règle métier : un étudiant ne peut pas avoir plus de 3 emprunts actifs ?
- Comment filtrer efficacement les informations liées aux prêts ?
 - les emprunts en cours (getActiveLoans),
 - les emprunts d'un étudiant donné (getLoansByStudent).

4. Le projet doit intégrer un Repository générique pour manipuler n'importe quel type d'entité possédant un identifiant, via des opérations de base :

- ajout,
- accès,
- recherche,
- suppression.

Cela répond à la problématique de **réutilisabilité** du code.

Pour les groupes les plus rapides ou les plus avancés, plusieurs extensions visent à rendre l'application plus réaliste :

- gestion des réservations (file d'attente d'étudiants pour un livre indisponible),
- détection et gestion des retards avec calcul de pénalités,
- création d'un menu interactif texte pour simuler un usage utilisateur.

Consignes de réalisation

1. Modélisation (Interfaces & Types)

Créer :

Interface Author

- id: number
- name: string
- birthYear?: number (optionnel)

Interface Book

- id: number
- title: string
- author: Author
- available: boolean
- categories: string[]

Types

- type BookCategory = "novel" | "history" | "science" | "poetry"
- type Role = "student" | "librarian" | "admin"
- type LoanStatus = "ongoing" | "returned"

Interface Loan

- book: Book
- student: Student
- date: Date
- status: LoanStatus

Créer une fonction fléchée :

```
isValidCategory(cat: BookCategory): boolean
```

POO (Classes & Héritage)

Créer :

Classe User

- firstName, lastName (public)
- _age: number (privé)
- Getter/setter age avec validation (≥ 0)
- Méthode getFullName()

Classe Student **extends** User

- Méthode study()

Classe Librarian **extends** User

- Méthode manage()

Créer un tableau User[] et démontrer le polymorphisme.

Gestion des livres (Classe Library)

Créer une classe Library avec :

- private books: Book[] = []
- addBook(book: Book): void
- listAvailable(): Book[]
- findBookById(id: number): Book | undefined

Ajouter 3 livres au système.

Gestion des emprunts (Loan)

Créer :

- 3 emprunts
- Une fonction :

```
getActiveLoans(loans: Loan[]): Loan[]
```


Repository générique

Créer un fichier repository.ts contenant :

```
export class Repository<T extends { id: number }> {  
  add(item: T): void  
  getAll(): T[]  
  findById(id: number): T | undefined  
  removeById(id: number): boolean  
}
```

Tester avec :

- Repository
- Repository

Organisation en modules

Arborescence obligatoire :

```
src/  
  author.ts  
  book.ts  
  user.ts  
  loan.ts  
  library.ts  
  repository.ts  
  main.ts
```

Scénario de démonstration (main.ts)

Dans main.ts :

- Créer auteurs, livres, utilisateurs, prêts
- Démontrer chaque fonctionnalité :
 - ajout / liste de livres
 - polymorphisme utilisateur
 - emprunts actifs
 - repository générique
- Afficher des résultats explicites dans la console.

Bonus facultatif

- Gestion des retards
- File de réservation
- Limite d'emprunts par étudiant
- Menu interactif texte

Livrables

- Projet complet compilable (tsc sans erreur)
- Organisation en modules
- Clair, propre, typé